Tic-Tac-Toe Console Game

Technical Specification

1. Project Overview

Project Name: TicTacToe
Platform: .NET 10.0
Language: C#
Interface: Console (text-based UI)

The goal is to implement a **two-player Tic-Tac-Toe game** playable in the console. The game displays a 3×3 grid and allows players to take turns entering positions until a win or draw occurs.

The application should follow **clean code practices and modular structure**, making it easy to extend later (e.g., AI player, GUI).

---

2. Functional Requirements

2.1 Game Rules

The game follows standard Tic-Tac-Toe rules:

- Two players:
  - Player 1 uses **X**
  - Player 2 uses **O**
- Players alternate turns.
- Each turn a player selects an empty cell.
- The first player to get **three marks in a row** wins.

Valid win combinations:

- Horizontal rows
- Vertical columns
- Diagonals

If the board fills with no winner, the game ends in a **draw**.

---

3. Game Flow

1. Game Start

Program starts and displays:

TIC TAC TOE

Player 1: X

Player 2: O

Then an empty board is displayed.

2. Player Turn Loop

For each turn:

1. Display current board
2. Show current player
3. Ask for input
4. Validate input
5. Update board
6. Check win condition
7. Check draw condition
8. Switch player

3. Game End

If a player wins:

Player X wins!

If draw:

The game is a draw.

Then prompt:

Play again? (Y/N)

---

4. Board Representation

Board size: **3×3**

Cell positions correspond to numeric keypad layout:

1 | 2 | 3

---

4 | 5 | 6

---

7 | 8 | 9

Internally the board should be represented as:

char[3,3]

or

char[]

Example internal state:

```
X | O | X
---------
  | X |
---------
O |   | O
```

Empty cells contain:

```
' '
```

---

5. Console UI Layout

The board must render clearly.

Example:

```
   |   |
 X | O | X
---+---+---
   | X |
---+---+---
 O |   | O
   |   |
```

Alternative minimal layout (acceptable):

```
X | O | X
---------
  | X |
---------
O |   | O
```

---

6. Player Input

Input is a **number 1-9** representing board position.

Example prompt:

Player X, choose a position (1-9):

Validations

The program must validate:

Condition	Behavior
Input not a number	Show error
Number not in 1-9	Show error
Cell already occupied	Show error

Example message:

Invalid move. Try again.

The player must retry until a valid move is entered.

---

7. Win Detection

After every move the game must check for a winner.

Winning patterns:

(0,0) (0,1) (0,2)

(1,0) (1,1) (1,2)

(2,0) (2,1) (2,2)

Columns

Rows

Diagonals

Example winning board:

```
X | X | X
---------
O | O |
---------
  |   |
```

---

8. Draw Detection

A draw occurs when:

- All **9 cells are filled**
- No winning combination exists

---

9. Non-Functional Requirements

Code Quality

Code must follow:

- SOLID principles where practical
- Separation of concerns
- Clean method naming
- No global mutable state

Performance

Performance requirements are minimal since the board is small.

Maintainability

The code should allow easy future additions:

- AI player
- Larger board sizes
- GUI version

---

10. Suggested Project Structure

```
TicTacToe.Console
|
 ├── Program.cs
 ├── Game
|   ├── GameController.cs
|   ├── GameBoard.cs
|   └── Player.cs
|
 ├── UI
|   └── ConsoleRenderer.cs
|
└── Logic
```

└── WinChecker.cs

---

11. Component Responsibilities

Program.cs

Application entry point.

Responsibilities:

- Initialize game
- Start game loop

GameController

Coordinates gameplay.

Responsibilities:

- Manage turns
- Handle player switching
- Call win/draw logic
- Communicate with UI

Key methods:

StartGame()

ProcessTurn()

SwitchPlayer()

ResetGame()

GameBoard

Represents the board state.

Responsibilities:

- Store board data
- Validate moves
- Apply moves

Key methods:

bool IsCellEmpty(int position)

void PlaceMark(int position, char mark)

bool IsFull()

void Reset()

Player

Represents a player.

Properties:

Name

Symbol (X or O)

WinChecker

Handles win detection.

Key method:

bool CheckWin(char[,] board, char symbol)

ConsoleRenderer

Handles all console output.

Responsibilities:

- Draw board
- Show messages
- Ask for input

Key methods:

RenderBoard()

ShowMessage(string message)

int GetPlayerMove()

---

12. Error Handling

The application must gracefully handle:

- Invalid numeric input
- Non-numeric input
- Occupied cells

The game must **never crash due to input errors**.

---

13. Game State Model

Game states:

NotStarted

Running

Win

Draw

---

14. Example Game Session

TIC TAC TOE

Player X turn

Choose position (1-9): 5

```
  |   |
---+---+---
  | X |
---+---+---
```

Player O turn

Choose position (1-9): 1

Game continues until completion.

---

15. Future Extension Possibilities (Optional)

The design should make it easy to add:

- AI opponent (Minimax algorithm)
- Score tracking
- Multiple rounds
- Colored console output
- Configurable board size
- Online multiplayer

---

16. Deliverables

The AI implementation should produce:

1. Fully functional console game
2. Cleanly structured C# project
3. Compilable with:

dotnet build

dotnet run

Target framework:

net10.0